The last requirement is one way of saying that the solution should be efficient; that is, it should allow the maximum amount of concurrency.

We make no assumption about how long `eat` and `think` take, except that `eat` has to terminate eventually. Otherwise, the third constraint is impossible—if a philosopher keeps one of the forks forever, nothing can prevent the neighbors from starving.

To make it easy for philosophers to refer to their forks, we can use the functions `left` and `right`:

Which fork?

```
def left(i): return i
def right(i): return (i + 1) % 5
```

The `%` operator wraps around when it gets to 5, so `(4 + 1) % 5 = 0`.

Since we have to enforce exclusive access to the forks, it is natural to use a list of Semaphores, one for each fork. Initially, all the forks are available.

Variables for dining philosophers

```
forks = [Semaphore(1) for i in range(5)]
```

This notation for initializing a list might be unfamiliar to readers who don't use Python. The `range` function returns a list with 5 elements; for each element of this list, Python creates a Semaphore with initial value 1 and assembles the result in a list named `forks`.

Here is an initial attempt at `get_fork` and `put_fork`:

Dining philosophers non-solution

```
def get_forks(i):
    fork[right(i)].wait()
    fork[left(i)].wait()

def put_forks(i):
    fork[right(i)].signal()
    fork[left(i)].signal()
```

It's clear that this solution satisfies the first constraint, but we can be pretty sure it doesn't satisfy the other two, because if it did, this wouldn't be an interesting problem and you would be reading Section 5.1.

Puzzle: what's wrong?

4.5.1 Deadlock #5

The problem is that the table is round. As a result, each philosopher can pick up a fork and then wait forever for the other fork. Deadlock!

Puzzle: write a solution to this problem that prevents deadlock.

Hint: one way to avoid deadlock is to think about the conditions that make deadlock possible and then change one of those conditions. In this case, the deadlock is fairly fragile—a very small change breaks it.

4.5.2 Dining philosophers hint #1

If only four philosophers are allowed at the table at a time, deadlock is impossible.

First, convince yourself that this claim is true, then write code that limits the number of philosophers at the table.

4.5.3 Dining philosophers solution #1

If there are only four philosophers at the table, then in the worst case each one picks up a fork. Even then, there is a fork left on the table, and that fork has two neighbors, each of which is holding another fork. Therefore, either of these neighbors can pick up the remaining fork and eat.

We can control the number of philosophers at the table with a Multiplex named `footman` that is initialized to 4. Then the solution looks like this:

Dining philosophers solution #1

```
def get_forks(i):
    footman.wait()
    fork[right(i)].wait()
    fork[left(i)].wait()

def put_forks(i):
    fork[right(i)].signal()
    fork[left(i)].signal()
    footman.signal()
```

In addition to avoiding deadlock, this solution also guarantees that no philosopher starves. Imagine that you are sitting at the table and both of your neighbors are eating. You are blocked waiting for your right fork. Eventually your right neighbor will put it down, because `eat` can't run forever. Since you are the only thread waiting for that fork, you will necessarily get it next. By a similar argument, you cannot starve waiting for your left fork.

Therefore, the time a philosopher can spend at the table is bounded. That implies that the wait time to get into the room is also bounded, as long as `footman` has Property 4 (see Section 4.4).

This solution shows that by controlling the number of philosophers, we can avoid deadlock. Another way to avoid deadlock is to change the order in which the philosophers pick up forks. In the original non-solution, the philosophers are "righties"; that is, they pick up the right fork first. But what happens if Philosopher 0 is a leftie?

Puzzle: prove that if there is at least one leftie and at least one rightie, then deadlock is not possible.

Hint: deadlock can only occur when all 5 philosophers are holding one fork and waiting, forever, for the other. Otherwise, one of them could get both forks, eat, and leave.

The proof works by contradiction. First, assume that deadlock is possible. Then choose one of the supposedly deadlocked philosophers. If she's a leftie, you can prove that the philosophers are all lefties, which is a contradiction. Similarly, if she's a rightie, you can prove that they are all righties. Either way you get a contradiction; therefore, deadlock is not possible.

4.5.4 Dining philosopher's solution #2

In the asymmetric solution to the Dining philosophers problem, there has to be at least one leftie and at least one rightie at the table. In that case, deadlock is impossible. The previous hint outlines the proof. Here are the details.

Again, if deadlock is possible, it occurs when all 5 philosophers are holding one fork and waiting for the other. If we assume that Philosopher j is a leftie, then she must be holding her left fork and waiting for her right. Therefore her neighbor to the right, Philosopher k, must be holding his left fork and waiting for his right neighbor; in other words, Philosopher k must be a leftie. Repeating the same argument, we can prove that the philosophers are all lefties, which contradicts the original claim that there is at least one rightie. Therefore deadlock is not possible.

The same argument we used for the previous solution also proves that starvation is impossible for this solution.

4.5.5 Tanenbaum's solution

There is nothing wrong with the previous solutions, but just for completeness, let's look at some alternatives. One of the best known is the one that appears in Tanenbaum's popular operating systems textbook [12]. For each philosopher there is a state variable that indicates whether the philosopher is thinking, eating, or waiting to eat ("hungry") and a semaphore that indicates whether the philosopher can start eating. Here are the variables:

Variables for Tanenbaum's solution

```
state = ['thinking'] * 5
sem = [Semaphore(0) for i in range(5)]
mutex = Semaphore(1)
```

The initial value of `state` is a list of 5 copies of `'thinking'`. `sem` is a list of 5 semaphores with the initial value 0. Here is the code:

Tanenbaum's solution

```
def get_fork(i):
    mutex.wait()
    state[i] = 'hungry'
    test(i)
    mutex.signal()
    sem[i].wait()

def put_fork(i):
    mutex.wait()
    state[i] = 'thinking'
    test(right(i))
    test(left(i))
    mutex.signal()

def test(i):
    if state[i] == 'hungry' and
    state[left (i)]  != 'eating' and
    state[right (i)] != 'eating':
        state[i] = 'eating'
        sem[i].signal()
```

The `test` function checks whether the ith philosopher can start eating, which he can if he is hungry and neither of his neighbors are eating. If so, the `test` signals semaphore i.

There are two ways a philosopher gets to eat. In the first case, the philosopher executes `get_forks`, finds the forks available, and proceeds immediately. In the second case, one of the neighbors is eating and the philosopher blocks on its own semaphore. Eventually, one of the neighbors will finish, at which point it executes `test` on both of its neighbors. It is possible that both tests

will succeed, in which case the neighbors can run concurrently. The order of the two tests doesn't matter.

In order to access `state` or invoke `test`, a thread has to hold `mutex`. Thus, the operation of checking and updating the array is atomic. Since a philosopher can only proceed when we know both forks are available, exclusive access to the forks is guaranteed.

No deadlock is possible, because the only semaphore that is accessed by more than one philosopher is `mutex`, and no thread executes `wait` while holding `mutex`.

But again, starvation is tricky.

Puzzle: Either convince yourself that Tanenbaum's solution prevents starvation or find a repeating pattern that allows a thread to starve while other threads make progress.

4.5.6 Starving Tanenbaums

Unfortunately, this solution is not starvation-free. Gingras demonstrated that there are repeating patterns that allow a thread to wait forever while other threads come and go [4].

Imagine that we are trying to starve Philosopher 0. Initially, 2 and 4 are at the table and 1 and 3 are hungry. Imagine that 2 gets up and 1 sits down; then 4 gets up and 3 sits down. Now we are in the mirror image of the starting position.

If 3 gets up and 4 sits down, and then 1 gets up and 2 sits down, we are back where we started. We could repeat the cycle indefinitely and Philosopher 0 would starve.

So, Tanenbaum's solution doesn't satisfy all the requirements.

4.6 Cigarette smokers problem

The cigarette smokers problem was originally presented by Suhas Patil [8], who claimed that it cannot be solved with semaphores. That claim comes with some qualifications, but in any case the problem is interesting and challenging.

Four threads are involved: an agent and three smokers. The smokers loop forever, first waiting for ingredients, then making and smoking cigarettes. The ingredients are tobacco, paper, and matches.

We assume that the agent has an infinite supply of all three ingredients, and each smoker has an infinite supply of one of the ingredients; that is, one smoker has matches, another has paper, and the third has tobacco.

The agent repeatedly chooses two different ingredients at random and makes them available to the smokers. Depending on which ingredients are chosen, the smoker with the complementary ingredient should pick up both resources and proceed.

For example, if the agent puts out tobacco and paper, the smoker with the matches should pick up both ingredients, make a cigarette, and then signal the agent.

To explain the premise, the agent represents an operating system that allocates resources, and the smokers represent applications that need resources. The problem is to make sure that if resources are available that would allow one or more applications to proceed, those applications should be woken up. Conversely, we want to avoid waking an application if it cannot proceed.

Based on this premise, there are three versions of this problem that often appear in textbooks:

The impossible version: Patil's version imposes restrictions on the solution. First, you are not allowed to modify the agent code. If the agent represents an operating system, it makes sense to assume that you don't want to modify it every time a new application comes along. The second restriction is that you can't use conditional statements or an array of semaphores. With these constraints, the problem cannot be solved, but as Parnas points out, the second restriction is pretty artificial [7]. With constraints like these, a lot of problems become unsolvable.

The interesting version: This version keeps the first restriction—you can't change the agent code—but it drops the others.

The trivial version: In some textbooks, the problem specifies that the agent should signal the smoker that should go next, according to the ingredients that are available. This version of the problem is uninteresting because it makes the whole premise, the ingredients and the cigarettes, irrelevant. Also, as a practical matter, it is probably not a good idea to require the agent to know about the other threads and what they are waiting for. Finally, this version of the problem is just too easy.

Naturally, we will focus on the interesting version. To complete the statement of the problem, we need to specify the agent code. The agent uses the following semaphores:

Agent semaphores

```
agentSem = Semaphore(1)
tobacco = Semaphore(0)
paper = Semaphore(0)
match = Semaphore(0)
```

The agent is actually made up of three concurrent threads, Agent A, Agent B and Agent C. Each waits on `agentSem`; each time `agentSem` is signaled, one of the Agents wakes up and provides ingredients by signaling two semaphores.

Agent A code

```
agentSem.wait()
tobacco.signal()
paper.signal()
```

Agent B code

```
agentSem.wait()
paper.signal()
match.signal()
```

Agent C code

```
agentSem.wait()
tobacco.signal()
match.signal()
```

This problem is hard because the natural solution does not work. It is tempting to write something like:

Smoker with matches

```
tobacco.wait()
paper.wait()
agentSem.signal()
```

Smoker with tobacco

```
paper.wait()
match.wait()
agentSem.signal()
```

Smoker with paper

```
tobacco.wait()
match.wait()
agentSem.signal()
```

What's wrong with this solution?

4.6.1 Deadlock #6

The problem with the previous solution is the possibility of deadlock. Imagine that the agent puts out tobacco and paper. Since the smoker with matches is waiting on `tobacco`, it might be unblocked. But the smoker with tobacco is waiting on `paper`, so it is possible (even likely) that it will also be unblocked. Then the first thread will block on `paper` and the second will block on `match`. Deadlock!

4.6.2 Smokers problem hint

The solution proposed by Parnas uses three helper threads called "pushers" that respond to the signals from the agent, keep track of the available ingredients, and signal the appropriate smoker.

The additional variables and semaphores are

Smokers problem hint

```
isTobacco = isPaper = isMatch = False
tobaccoSem = Semaphore(0)
paperSem = Semaphore(0)
matchSem = Semaphore(0)
```

The boolean variables indicate whether or not an ingredient is on the table. The pushers use `tobaccoSem` to signal the smoker with tobacco, and the other semaphores likewise.

4.6.3 Smoker problem solution

Here is the code for one of the pushers:

Pusher A

```
tobacco.wait()
mutex.wait()
    if isPaper:
        isPaper = False
        matchSem.signal()
    elif isMatch:
        isMatch = False
        paperSem.signal()
    else:
        isTobacco = True
mutex.signal()
```

This pusher wakes up any time there is tobacco on the table. If it finds `isPaper` true, it knows that Pusher B has already run, so it can signal the smoker with matches. Similarly, if it finds a match on the table, it can signal the smoker with paper.

But if Pusher A runs first, then it will find both `isPaper` and `isMatch` false. It cannot signal any of the smokers, so it sets `isTobacco`.

The other pushers are similar. Since the pushers do all the real work, the smoker code is trivial:

Smoker with tobacco

```
tobaccoSem.wait()
makeCigarette()
agentSem.signal()
smoke()
```

Parnas presents a similar solution that assembles the boolean variables, bitwise, into an integer, and then uses the integer as an index into an array of semaphores. That way he can avoid using conditionals (one of the artificial constraints). The resulting code is a bit more concise, but its function is not as obvious.

4.7 Generalized Smokers Problem

Parnas suggested that the smokers problem becomes more difficult if we modify the agent, eliminating the requirement that the agent wait after putting out ingredients. In this case, there might be multiple instances of an ingredient on the table.

Puzzle: modify the previous solution to deal with this variation.

4.7.1 Generalized Smokers Problem Hint

If the agents don't wait for the smokers, ingredients might accumulate on the table. Instead of using boolean values to keep track of ingredients, we need integers to count them.

Generalized Smokers problem hint

```
1 numTobacco = numPaper = numMatch = 0
```

4.7.2 Generalized Smokers Problem Solution

Here is the modified code for Pusher A:

Pusher A

```
tobacco.wait()
mutex.wait()
    if numPaper:
        numPaper -= 1
        matchSem.signal()
    elif numMatch:
        numMatch -= 1
        paperSem.signal()
    else:
        numTobacco += 1
mutex.signal()
```

One way to visualize this problem is to imagine that when an Agent runs, it creates two pushers, gives each of them one ingredient, and puts them in a room with all the other pushers. Because of the mutex, the pushers file into a room where there are three sleeping smokers and a table. One at a time, each pusher enters the room and checks the ingredients on the table. If he can assemble a complete set of ingredients, he takes them off the table and wakes the corresponding smoker. If not, he leaves his ingredient on the table and leaves without waking anyone.

This is an example of a pattern we will see several times, which I call a **scoreboard**. The variables `numPaper`, `numTobacco` and `numMatch` keep track of the state of the system. As each thread files through the mutex, it checks the state as if looking at the scoreboard, and reacts accordingly.

Chapter 5

Less classical synchronization problems

5.1 The dining savages problem

This problem is from Andrews's *Concurrent Programming* [1].

> A tribe of savages eats communal dinners from a large pot that can hold M servings of stewed missionary[1]. When a savage wants to eat, he helps himself from the pot, unless it is empty. If the pot is empty, the savage wakes up the cook and then waits until the cook has refilled the pot.

Any number of savage threads run the following code:

Unsynchronized savage code

```
while True:
    getServingFromPot()
    eat()
```

And one cook thread runs this code:

Unsynchronized cook code

```
while True:
    putServingsInPot(M)
```

[1]This problem is based on a cartoonish representation of the history of Western missionaries among hunter-gatherer societies. Some humor is intended by the allusion to the Dining Philosophers problem, but the representation of "savages" here isn't intended to be any more realistic than the previous representation of philosophers. If you are interested in hunter-gatherer societies, I recommend Jared Diamond's *Guns, Germs and Steel*, Napoleon Chagnon's *The Yanomamo*, and Redmond O'Hanlon's *In Trouble Again*, but not Tierney's *Darkness in El Dorado*, which I believe is unreliable.

The synchronization constraints are:

- Savages cannot invoke `getServingFromPot` if the pot is empty.
- The cook can invoke `putServingsInPot` only if the pot is empty.

Puzzle: Add code for the savages and the cook that satisfies the synchronization constraints.

5.1.1 Dining Savages hint

It is tempting to use a semaphore to keep track of the number of servings, as in the producer-consumer problem. But in order to signal the cook when the pot is empty, a thread would have to know before decrementing the semaphore whether it would have to wait, and we just can't do that.

An alternative is to use a scoreboard to keep track of the number of servings. If a savage finds the counter at zero, he wakes the cook and waits for a signal that the pot is full. Here are the variables I used:

Dining Savages hint

```
servings = 0
mutex = Semaphore(1)
emptyPot = Semaphore(0)
fullPot = Semaphore(0)
```

Not surprisingly, `emptyPot` indicates that the pot is empty and `fullPot` indicates that the pot is full.

5.1.2 Dining Savages solution

My solution is a combination of the scoreboard pattern with a rendezvous. Here is the code for the cook:

Dining Savages solution (cook)

```
while True:
    emptyPot.wait()
    putServingsInPot(M)
    fullPot.signal()
```

The code for the savages is only a little more complicated. As each savage passes through the mutex, he checks the pot. If it is empty, he signals the cook and waits. Otherwise, he decrements `servings` and gets a serving from the pot.

Dining Savages solution (savage)

```
while True:
    mutex.wait()
        if servings == 0:
            emptyPot.signal()
            fullPot.wait()
            servings = M
        servings -= 1
        getServingFromPot()
    mutex.signal()

    eat()
```

It might seem odd that the savage, rather than the cook, sets `servings = M`. That's not really necessary; when the cook runs `putServingsInPot`, we know that the savage that holds the mutex is waiting on `fullPot`. So the cook could access `servings` safely. But in this case, I decided to let the savage do it so that it is clear from looking at the code that all accesses to `servings` are inside the mutex.

This solution is deadlock-free. The only opportunity for deadlock comes when the savage that holds `mutex` waits for `fullPot`. While he is waiting, other savages are queued on `mutex`. But eventually the cook will run and signal `fullPot`, which allows the waiting savage to resume and release the mutex.

Does this solution assume that the pot is thread-safe, or does it guarantee that `putServingsInPot` and `getServingFromPot` are executed exclusively?

5.2 The barbershop problem

The original barbershop problem was proposed by Dijkstra. A variation of it appears in Silberschatz and Galvin's *Operating Systems Concepts* [10].

> A barbershop consists of a waiting room with n chairs, and the barber room containing the barber chair. If there are no customers to be served, the barber goes to sleep. If a customer enters the barbershop and all chairs are occupied, then the customer leaves the shop. If the barber is busy, but chairs are available, then the customer sits in one of the free chairs. If the barber is asleep, the customer wakes up the barber. Write a program to coordinate the barber and the customers.

To make the problem a little more concrete, I added the following information:

- Customer threads should invoke a function named `getHairCut`.
- If a customer thread arrives when the shop is full, it can invoke `balk`, which does not return.
- The barber thread should invoke `cutHair`.
- When the barber invokes `cutHair` there should be exactly one thread invoking `getHairCut` concurrently.

Write a solution that guarantees these constraints.

5.2.1 Barbershop hint

Barbershop hint

```
n = 4
customers = 0
mutex = Semaphore(1)
customer = Semaphore(0)
barber = Semaphore(0)
customerDone = Semaphore(0)
barberDone = Semaphore(0)
```

`n` is the total number of customers that can be in the shop: three in the waiting room and one in the chair.

`customers` counts the number of customers in the shop; it is protected by `mutex`.

The barber waits on `customer` until a customer enters the shop, then the customer waits on `barber` until the barber signals him to take a seat.

After the haircut, the customer signals `customerDone` and waits on `barberDone`.

5.2.2 Barbershop solution

This solution combines a scoreboard and two rendezvouses[2]. Here is the code for customers:

Barbershop solution (customer)

```
mutex.wait()
    if customers == n:
        mutex.signal()
        balk()
    customers += 1
mutex.signal()

customer.signal()
barber.wait()

# getHairCut()

customerDone.signal()
barberDone.wait()

mutex.wait()
    customers -= 1
mutex.signal()
```

If there are n customers in the shop any customers that arrive immediately invoke `balk`. Otherwise each customer signals `customer` and waits on `barber`.

Here is the code for the barber:

Barbershop solution (barber)

```
customer.wait()
barber.signal()

# cutHair()

customerDone.wait()
barberDone.signal()
```

Each time a customer signals, the barber wakes, signals `barber`, and gives one hair cut. If another customer arrives while the barber is busy, then on the next iteration the barber will pass the `customer` semaphore without sleeping.

The names for `customer` and `barber` are based on the naming convention for a rendezvous, so `customer.wait()` means "wait for a customer," not "customers wait here."

[2]The plural of rendezvous is rare, and not all dictionaries agree about what it is. Another possibility is that the plural is also spelled "rendezvous," but the final "s" is pronounced.

The second rendezvous, using `customerDone` and `barberDone`, ensures that the hair cut is done before the barber loops around to let the next customer into the critical section.

This solution is in `sync_code/barber.py` (see 3.2).

5.3 The FIFO barbershop

In the previous solution there is no guarantee that customers are served in the order they arrive. Up to `n` customers can pass the turnstile, signal `customer` and wait on `barber`. When the barber signal `barber`, any of the customers might proceed.

Modify this solution so that customers are served in the order they pass the turnstile.

Hint: you can refer to the current thread as `self`, so if you write `self.sem = Semaphore(0)`, each thread gets its own semaphore.

5.3.1 FIFO barbershop hint

My solution uses a list of semaphores named `queue`.

FIFO barbershop hint

```
n = 4
customers = 0
mutex = Semaphore(1)
customer = Semaphore(0)
customerDone = Semaphore(0)
barberDone = Semaphore(0)
queue = []
```

As each thread passes the turnstile, it creates a semaphore and puts it in the queue.

Instead of waiting on `barber`, each thread waits on its own semaphore. When the barber wakes up, he removes a thread from the queue and signals it.

5.3.2 FIFO barbershop solution

Here is the modified code for customers:

FIFO barbershop solution (customer)

```
self.sem = Semaphore(0)
mutex.wait()
    if customers == n:
        mutex.signal()
        balk()
    customers += 1
    queue.append(self.sem)
mutex.signal()

customer.signal()
self.sem.wait()

# getHairCut()

customerDone.signal()
barberDone.wait()

mutex.wait()
    customers -= 1
mutex.signal()
```

And the code for barbers:

FIFO barbershop solution (barber)

```
customer.wait()
mutex.wait()
    sem = queue.pop(0)
mutex.signal()

sem.signal()

# cutHair()

customerDone.wait()
barberDone.signal()
```

Notice that the barber has to get `mutex` to access the queue.
This solution is in `sync_code/barber2.py` (see 3.2).

5.4 Hilzer's Barbershop problem

William Stallings [11] presents a more complicated version of the barbershop problem, which he attributes to Ralph Hilzer at the California State University at Chico.

> Our barbershop has three chairs, three barbers, and a waiting area that can accommodate four customers on a sofa and that has standing room for additional customers. Fire codes limit the total number of customers in the shop to 20.
>
> A customer will not enter the shop if it is filled to capacity with other customers. Once inside, the customer takes a seat on the sofa or stands if the sofa is filled. When a barber is free, the customer that has been on the sofa the longest is served and, if there are any standing customers, the one that has been in the shop the longest takes a seat on the sofa. When a customer's haircut is finished, any barber can accept payment, but because there is only one cash register, payment is accepted for one customer at a time. The barbers divide their time among cutting hair, accepting payment, and sleeping in their chair waiting for a customer.

In other words, the following synchronization constraints apply:

- Customers invoke the following functions in order: `enterShop`, `sitOnSofa`, `getHairCut`, and `pay`.
- Barbers invoke `cutHair` and `acceptPayment`.
- Customers cannot invoke `enterShop` if the shop is at capacity.
- If the sofa is full, an arriving customer cannot invoke `sitOnSofa`.
- When a customer invokes `getHairCut` there should be a corresponding barber executing `cutHair` concurrently, and vice versa.
- It should be possible for up to three customers to execute `getHairCut` concurrently, and up to three barbers to execute `cutHair` concurrently.
- The customer has to `pay` before the barber can `acceptPayment`.
- The barber must `acceptPayment` before the customer can exit.

Puzzle: Write code that enforces the synchronization constraints for Hilzer's barbershop.

5.4.1 Hilzer's barbershop hint

Here are the variables I used in my solution:

Hilzer's barbershop hint

```
n = 20
customers = 0
mutex = Semaphore(1)
sofa = Semaphore(4)
customer1 = Semaphore(0)
customer2 = Semaphore(0)
barber = Semaphore(0)
payment = Semaphore(0)
receipt = Semaphore(0)
queue1 = []
queue2 = []
```

`mutex` protects `customers`, which keeps track of the number of customers in the shop, and `queue1` which is a list of semaphores for threads waiting for a seat on the sofa.

`mutex2` protects `queue2`, which is a list of semaphores for threads waiting for a chair.

`sofa` is a multiplex that enforces the maximum number of customers on the sofa.

`customer1` signals that there is a customer in `queue1`, and `customer2` signals that there is a customer in `queue2`.

`payment` signals that a customer has paid, and `receipt` sigmals that a barber has accepted payment.

5.4.2 Hilzer's barbershop solution

This solution is considerably more complex than I expected. I am not sure if Hilzer had something simpler in mind, but here is the best I could do.

Hilzer's barbershop solution (customer)

```
self.sem1 = Semaphore(0)
self.sem2 = Semaphore(0)

mutex.wait()
    if customers == n:
        mutex.signal()
        balk()
    customers += 1
    queue1.append(self.sem1)
mutex.signal()

# enterShop()
customer1.signal()
self.sem1.wait()

sofa.wait()
    # sitOnSofa()
    self.sem1.signal()
    mutex.wait()
        queue2.append(self.sem2)
    mutex.signal()
    customer2.signal()
    self.sem2.wait()
sofa.signal()

# sitInBarberChair()

# pay()
payment.signal()
receipt.wait()

mutex.wait()
    customers -= 1
mutex.signal()
```

The first paragraph is the same as in the previous solution. When a customer arrives, it checks the counter and either balks or adds itself to the queue. Then it signals a barber.

When the customer gets out of queue, it enters the multiplex, sits on the couch and adds itself to the second queue.

When it gets out of *that* queue, it gets a haircut, pays, and then exits.

Hilzer's barbershop solution (barber)

```
customer1.wait()
mutex.wait()
    sem = queue1.pop(0)
    sem.signal()
    sem.wait()
mutex.signal()
sem.signal()

customer2.wait()
mutex.wait()
    sem = queue2.pop(0)
mutex.signal()
sem.signal()

barber.signal()
# cutHair()

payment.wait()
# acceptPayment()
receipt.signal()
```

Each barber waits for a customer to enter, signals the customer's semaphore to get it out of queue, then waits for it to claim a seat on the sofa. This enforces the FIFO requirement.

The barber waits for the customer to join the second queue and then signals it, allowing the customer to claim a chair.

Each barber admits one customer to the chair, so there can by up to three concurrent haircuts. Because there is only one cash register, the customer has to get `mutex`. The customer and barber rendezvous at the cash register, then both exit.

This solution satisfies the synchonization constraints, but it leaves the sofa underutilized. Because there are only three barbers, there can never be more than three customers on the sofa, so the multiplex is unnecessary.

This solution is in `sync_code/barber3.py` (see 3.2).

The only way I can think of to solve this problem is to create a third kind of thread, which I'll call an usher. The ushers manage `queue1` and the barbers manage `queue2`. If there are 4 ushers and 3 barbers, the sofa can be fully utilized.

This solution is in `sync_code/barber4.py` (see 3.2).

5.5 The Santa Claus problem

This problem is from William Stallings's *Operating Systems* [11], but he attributes it to John Trono of St. Michael's College in Vermont.

> Santa Claus sleeps in his shop at the North Pole and can only be awakened by either (1) all nine reindeer being back from their vacation in the South Pacific, or (2) some of the elves having difficulty making toys; to allow Santa to get some sleep, the elves can only wake him when three of them have problems. When three elves are having their problems solved, any other elves wishing to visit Santa must wait for those elves to return. If Santa wakes up to find three elves waiting at his shop's door, along with the last reindeer having come back from the tropics, Santa has decided that the elves can wait until after Christmas, because it is more important to get his sleigh ready. (It is assumed that the reindeer do not want to leave the tropics, and therefore they stay there until the last possible moment.) The last reindeer to arrive must get Santa while the others wait in a warming hut before being harnessed to the sleigh.

Here are some addition specifications:

- After the ninth reindeer arrives, Santa must invoke `prepareSleigh`, and then all nine reindeer must invoke `getHitched`.
- After the third elf arrives, Santa must invoke `helpElves`. Concurrently, all three elves should invoke `getHelp`.
- All three elves must invoke `getHelp` before any additional elves enter (increment the elf counter).

Santa should run in a loop so he can help many sets of elves. We can assume that there are exactly 9 reindeer, but there may be any number of elves.

5.5.1 Santa problem hint

Santa problem hint

```
elves = 0
reindeer = 0
santaSem = Semaphore(0)
reindeerSem = Semaphore(0)
elfTex = Semaphore(1)
mutex = Semaphore(1)
```

`elves` and `reindeer` are counters, both protected by `mutex`. Elves and reindeer get `mutex` to modify the counters; Santa gets it to check them.

Santa waits on `santaSem` until either an elf or a reindeer signals him.

The reindeer wait on `reindeerSem` until Santa signals them to enter the paddock and get hitched.

The elves use `elfTex` to prevent additional elves from entering while three elves are being helped.

5.5.2 Santa problem solution

Santa's code is pretty straightforward. Remember that it runs in a loop.

Santa problem solution (Santa)

```
santaSem.wait()
mutex.wait()
    if reindeer >= 9:
        prepareSleigh()
        reindeerSem.signal(9)
        reindeer -= 9
    else if elves == 3:
        helpElves()
mutex.signal()
```

When Santa wakes up, he checks which of the two conditions holds and either deals with the reindeer or the waiting elves. If there are nine reindeer waiting, Santa invokes `prepareSleigh`, then signals `reindeerSem` nine times, allowing the reindeer to invoke `getHitched`. If there are elves waiting, Santa just invokes `helpElves`. There is no need for the elves to wait for Santa; once they signal `santaSem`, they can invoke `getHelp` immediately.

Santa doesn't have to decrement the `elves` counter because the elves do it on their way out.

Here is the code for reindeer:

Santa problem solution (reindeer)

```
mutex.wait()
    reindeer += 1
    if reindeer == 9:
        santaSem.signal()
mutex.signal()

reindeerSem.wait()
getHitched()
```

The ninth reindeer signals Santa and then joins the other reindeer waiting on `reindeerSem`. When Santa signals, the reindeer all execute `getHitched`.

The elf code is similar, except that when the third elf arrives it has to bar subsequent arrivals until the first three have executed `getHelp`.

Santa problem solution (elves)

```
elfTex.wait()
mutex.wait()
    elves += 1
    if elves == 3:
        santaSem.signal()
    else
        elfTex.signal()
mutex.signal()

getHelp()

mutex.wait()
    elves -= 1
    if elves == 0:
       elfTex.signal()
mutex.signal()
```

The first two elves release `elfTex` at the same time they release the `mutex`, but the last elf holds `elfTex`, barring other elves from entering until all three elves have invoked `getHelp`.

The last elf to leave releases `elfTex`, allowing the next batch of elves to enter.

5.6 Building H_2O

This problem has been a staple of the Operating Systems class at U.C. Berkeley for at least a decade. It seems to be based on an exercise in Andrews's *Concurrent Programming* [1].

There are two kinds of threads, oxygen and hydrogen. In order to assemble these threads into water molecules, we have to create a barrier that makes each thread wait until a complete molecule is ready to proceed.

As each thread passes the barrier, it should invoke `bond`. You must guarantee that all the threads from one molecule invoke `bond` before any of the threads from the next molecule do.

In other words:

- If an oxygen thread arrives at the barrier when no hydrogen threads are present, it has to wait for two hydrogen threads.
- If a hydrogen thread arrives at the barrier when no other threads are present, it has to wait for an oxygen thread and another hydrogen thread.

We don't have to worry about matching the threads up explicitly; that is, the threads do not necessarily know which other threads they are paired up with. The key is just that threads pass the barrier in complete sets; thus, if we examine the sequence of threads that invoke `bond` and divide them into groups of three, each group should contain one oxygen and two hydrogen threads.

Puzzle: Write synchronization code for oxygen and hydrogen molecules that enforces these constraints.

5.6.1 H_2O hint

Here are the variables I used in my solution:

Water building hint

```
mutex = Semaphore(1)
oxygen = 0
hydrogen = 0
barrier = Barrier(3)
oxyQueue = Semaphore(0)
hydroQueue = Semaphore(0)
```

`oxygen` and `hydrogen` are counters, protected by `mutex`. `barrier` is where each set of three threads meets after invoking `bond` and before allowing the next set of threads to proceed.

`oxyQueue` is the semaphore oxygen threads wait on; `hydroQueue` is the semaphore hydrogen threads wait on. I am using the naming convention for queues, so `oxyQueue.wait()` means "join the oxygen queue" and `oxyQueue.signal()` means "release an oxygen thread from the queue."

5.6.2 H_2O solution

Initially `hydroQueue` and `oxyQueue` are locked. When an oxygen thread arrives it signals `hydroQueue` twice, allowing two hydrogens to proceed. Then the oxygen thread waits for the hydrogen threads to arrive.

Oxygen code

```
mutex.wait()
oxygen += 1
if hydrogen >= 2:
    hydroQueue.signal(2)
    hydrogen -= 2
    oxyQueue.signal()
    oxygen -= 1
else:
    mutex.signal()

oxyQueue.wait()
bond()

barrier.wait()
mutex.signal()
```

As each oxygen thread enters, it gets the mutex and checks the scoreboard. If there are at least two hydrogen threads waiting, it signals two of them and itself and then bonds. If not, it releases the mutex and waits.

After bonding, threads wait at the barrier until all three threads have bonded, and then the oxygen thread releases the mutex. Since there is only one oxygen thread in each set of three, we are guaranteed to signal `mutex` once.

The code for hydrogen is similar:

Hydrogen code

```
mutex.wait()
hydrogen += 1
if hydrogen >= 2 and oxygen >= 1:
    hydroQueue.signal(2)
    hydrogen -= 2
    oxyQueue.signal()
    oxygen -= 1
else:
    mutex.signal()

hydroQueue.wait()
bond()

barrier.wait()
```

An unusual feature of this solution is that the exit point of the mutex is ambiguous. In some cases, threads enter the mutex, update the counter, and exit the mutex. But when a thread arrives that forms a complete set, it has to keep the mutex in order to bar subsequent threads until the current set have invoked `bond`.

After invoking `bond`, the three threads wait at a barrier. When the barrier opens, we know that all three threads have invoked `bond` and that one of them holds the mutex. We don't know *which* thread holds the mutex, but it doesn't matter as long as only one of them releases it. Since we know there is only one oxygen thread, we make it do the work.

This might seem wrong, because until now it has generally been true that a thread has to hold a lock in order to release it. But there is no rule that says that has to be true. This is one of those cases where it can be misleading to think of a mutex as a token that threads acquire and release.

5.7 River crossing problem

This is from a problem set written by Anthony Joseph at U.C. Berkeley, but I don't know if he is the original author. It is similar to the H_2O problem in the sense that it is a peculiar sort of barrier that only allows threads to pass in certain combinations.

Somewhere near Redmond, Washington there is a rowboat that is used by both Linux hackers and Microsoft employees (serfs) to cross a river. The ferry can hold exactly four people; it won't leave the shore with more or fewer. To guarantee the safety of the passengers, it is not permissible to put one hacker in the boat with three serfs, or to put one serf with three hackers. Any other combination is safe.

As each thread boards the boat it should invoke a function called `board`. You must guarantee that all four threads from each boatload invoke `board` before any of the threads from the next boatload do.

After all four threads have invoked `board`, exactly one of them should call a function named `rowBoat`, indicating that that thread will take the oars. It doesn't matter which thread calls the function, as long as one does.

Don't worry about the direction of travel. Assume we are only interested in traffic going in one of the directions.

5.7.1 River crossing hint

Here are the variables I used in my solution:

River crossing hint

```
barrier = Barrier(4)
mutex = Semaphore(1)
hackers = 0
serfs = 0
hackerQueue = Semaphore(0)
serfQueue = Semaphore(0)
local isCaptain = False
```

`hackers` and `serfs` count the number of hackers and serfs waiting to board. Since they are both protected by `mutex`, we can check the condition of both variables without worrying about an untimely update. This is another example of a scoreboard.

`hackerQueue` and `serfQueue` allow us to control the number of hackers and serfs that pass. The barrier makes sure that all four threads have invoked `board` before the captain invokes `rowBoat`.

`isCaptain` is a local variable that indicates which thread should invoke `rowBoat`.

5.7.2 River crossing solution

The basic idea of this solution is that each arrival updates one of the counters and then checks whether it makes a full complement, either by being the fourth of its kind or by completing a mixed pair of pairs.

I'll present the code for hackers; the serf code is symmetric (except, of course, that it is 1000 times bigger, full of bugs, and it contains an embedded web browser):

River crossing solution

```
mutex.wait()
    hackers += 1
    if hackers == 4:
        hackerQueue.signal(4)
        hackers = 0
        isCaptain = True
    elif hackers == 2 and serfs >= 2:
        hackerQueue.signal(2)
        serfQueue.signal(2)
        serfs -= 2
        hackers = 0
        isCaptain = True
    else:
        mutex.signal()      # captain keeps the mutex

hackerQueue.wait()

board()
barrier.wait()

if isCaptain:
    rowBoat()
    mutex.signal()          # captain releases the mutex
```

As each thread files through the mutual exclusion section, it checks whether a complete crew is ready to board. If so, it signals the appropriate threads, declares itself captain, and holds the mutex in order to bar additional threads until the boat has sailed.

The barrier keeps track of how many threads have boarded. When the last thread arrives, all threads proceed. The captain invoked `rowBoat` and then (finally) releases the mutex.

5.8 The roller coaster problem

This problem is from Andrews's *Concurrent Programming* [1], but he attributes it to J. S. Herman's Master's thesis.

> Suppose there are n passenger threads and a car thread. The passengers repeatedly wait to take rides in the car, which can hold C passengers, where $C < n$. The car can go around the tracks only when it is full.

Here are some additional details:

- Passengers should invoke `board` and `unboard`.
- The car should invoke `load`, `run` and `unload`.
- Passengers cannot board until the car has invoked `load`
- The car cannot depart until C passengers have boarded.
- Passengers cannot unboard until the car has invoked `unload`.

Puzzle: Write code for the passengers and car that enforces these constraints.

5.8.1 Roller Coaster hint

Roller Coaster hint

```
mutex = Semaphore(1)
mutex2 = Semaphore(1)
boarders = 0
unboarders = 0
boardQueue = Semaphore(0)
unboardQueue = Semaphore(0)
allAboard = Semaphore(0)
allAshore = Semaphore(0)
```

`mutex` protects `passengers`, which counts the number of passengers that have invoked `board`.

Passengers wait on `boardQueue` before boarding and `unboardQueue` before unboarding. `allAboard` indicates that the car is full.

5.8.2 Roller Coaster solution

Here is my code for the car thread:

Roller Coaster solution (car)

```
load()
boardQueue.signal(C)
allAboard.wait()

run()

unload()
unboardQueue.signal(C)
allAshore.wait()
```

When the car arrives, it signals C passengers, then waits for the last one to signal `allAboard`. After it departs, it allows C passengers to disembark, then waits for `allAshore`.

Roller Coaster solution (passenger)

```
boardQueue.wait()
board()

mutex.wait()
   boarders += 1
   if boarders == C:
       allAboard.signal()
       boarders = 0
mutex.signal()

unboardQueue.wait()
unboard()

mutex2.wait()
   unboarders += 1
   if unboarders == C:
       allAshore.signal()
       unboarders = 0
mutex2.signal()
```

Passengers wait for the car before boarding, naturally, and wait for the car to stop before leaving. The last passenger to board signals the car and resets the passenger counter.

5.9 Multi-car Roller Coaster problem

This solution does not generalize to the case where there is more than one car. In order to do that, we have to satisfy some additional constraints:

- Only one car can be boarding at a time.
- Multiple cars can be on the track concurrently.
- Since cars can't pass each other, they have to unload in the same order they boarded.
- All the threads from one carload must disembark before any of the threads from subsequent carloads.

Puzzle: modify the previous solution to handle the additional constraints. You can assume that there are m cars, and that each car has a local variable named `i` that contains an identifier between 0 and $m - 1$.

5.9.1 Multi-car Roller Coaster hint

I used two lists of semaphores to keep the cars in order. One represents the loading area and one represents the unloading area. Each list contains one semaphore for each car. At any time, only one semaphore in each list is unlocked, so that enforces the order threads can load and unload. Initially, only the semaphores for Car 0 are unlocked. As each car enters the loading (or unloading) it waits on its own semaphore; as it leaves it signals the next car in line.

Multi-car Roller Coaster hint

```
loadingArea = [Semaphore(0) for i in range(m)]
loadingArea[0].signal()
unloadingArea = [Semaphore(0) for i in range(m)]
unloadingArea[0].signal()
```

The function `next` computes the identifier of the next car in the sequence (wrapping around from $m-1$ to 0):

Implementation of `next`

```
def next(i):
    return (i + 1) % m
```

5.9.2 Multi-car Roller Coaster solution

Here is the modified code for the cars:

Multi-car Roller Coaster solution (car)

```
loadingArea[i].wait()
load()
boardQueue.signal(C)
allAboard.wait()
loadingArea[next(i)].signal()

run()

unloadingArea[i].wait()
unload()
unboardQueue.signal(C)
allAshore.wait()
unloadingArea[next(i)].signal()
```

The code for the passengers is unchanged.

Chapter 6

Not-so-classical problems

6.1 The search-insert-delete problem

This one is from Andrews's *Concurrent Programming* [1].

> Three kinds of threads share access to a singly-linked list: searchers, inserters and deleters. Searchers merely examine the list; hence they can execute concurrently with each other. Inserters add new items to the end of the list; insertions must be mutually exclusive to preclude two inserters from inserting new items at about the same time. However, one insert can proceed in parallel with any number of searches. Finally, deleters remove items from anywhere in the list. At most one deleter process can access the list at a time, and deletion must also be mutually exclusive with searches and insertions.

Puzzle: write code for searchers, inserters and deleters that enforces this kind of three-way categorical mutual exclusion.

6.1.1 Search-Insert-Delete hint

Search-Insert-Delete hint

```
1 insertMutex = Semaphore(1)
2 noSearcher = Semaphore(1)
3 noInserter = Semaphore(1)
4 searchSwitch = Lightswitch()
5 insertSwitch = Lightswitch()
```

`insertMutex` ensures that only one inserter is in its critical section at a time. `noSearcher` and `noInserter` indicate (surprise) that there are no searchers and no inserters in their critical sections; a deleter needs to hold both of these to enter.

`searchSwitch` and `insertSwitch` are used by searchers and inserters to exclude deleters.

6.1.2 Search-Insert-Delete solution

Here is my solution:

Search-Insert-Delete solution (searcher)

```
searchSwitch.wait(noSearcher)
# critical section
searchSwitch.signal(noSearcher)
```

The only thing a searcher needs to worry about is a deleter. The first searcher in takes `noSearcher`; the last one out releases it.

Search-Insert-Delete solution (inserter)

```
insertSwitch.wait(noInserter)
insertMutex.wait()
# critical section
insertMutex.signal()
insertSwitch.signal(noInserter)
```

Similarly, the first inserter takes `noInserter` and the last one out releases it. Since searchers and inserters compete for different semaphores, they can be in their critical section concurrently. But `insertMutex` ensures that only one inserter is in the room at a time.

Search-Insert-Delete solution (deleter)

```
noSearcher.wait()
noInserter.wait()
# critical section
noInserter.signal()
noSearcher.signal()
```

Since the deleter holds both `noSearcher` and `noInserter`, it is guaranteed exclusive access. Of course, any time we see a thread holding more than one semaphore, we need to check for deadlocks. By trying out a few scenarios, you should be able to convince yourself that this solution is deadlock free.

On the other hand, like many categorical exclusion problems, this one is prone to starvation. As we saw in the Readers-Writers problem, we can sometimes mitigate this problem by giving priority to one category of threads according to application-specific criteria. But in general it is difficult to write an efficient solution (one that allows the maximum degree of concurrency) that avoids starvation.

6.2 The unisex bathroom problem

I wrote this problem[1] when a friend of mine left her position teaching physics at Colby College and took a job at Xerox.

She was working in a cubicle in the basement of a concrete monolith, and the nearest women's bathroom was two floors up. She proposed to the Uberboss that they convert the men's bathroom on her floor to a unisex bathroom, sort of like on Ally McBeal.

The Uberboss agreed, provided that the following synchronization constraints can be maintained:

- There cannot be men and women in the bathroom at the same time.
- There should never be more than three employees squandering company time in the bathroom.

Of course the solution should avoid deadlock. For now, though, don't worry about starvation. You may assume that the bathroom is equipped with all the semaphores you need.

[1] Later I learned that a nearly identical problem appears in Andrews's *Concurrent Programming*[1]

6.2.1 Unisex bathroom hint

Here are the variables I used in my solution:

Unisex bathroom hint

```
empty = Semaphore(1)
maleSwitch = Lightswitch()
femaleSwitch = Lightswitch()
maleMultiplex = Semaphore(3)
femaleMultiplex = Semaphore(3)
```

`empty` is 1 if the room is empty and 0 otherwise.

`maleSwitch` allows men to bar women from the room. When the first male enters, the lightswitch locks `empty`, barring women; When the last male exits, it unlocks `empty`, allowing women to enter. Women do likewise using `femaleSwitch`.

`maleMultiplex` and `femaleMultiplex` ensure that there are no more than three men and three women in the system at a time.

6.2.2 Unisex bathroom solution

Here is the female code:

Unisex bathroom solution (female)

```
femaleSwitch.lock(empty)
    femaleMultiplex.wait()
        # bathroom code here
    femaleMultiplex.signal()
female Switch.unlock(empty)
```

The male code is similar.

Are there any problems with this solution?

6.3 No-starve unisex bathroom problem

The problem with the previous solution is that it allows starvation. A long line of women can arrive and enter while there is a man waiting, and vice versa.

Puzzle: fix the problem.

6.3.1 No-starve unisex bathroom solution

As we have seen before, we can use a turnstile to allow one kind of thread to stop the flow of the other kind of thread. This time we'll look at the male code:

No-starve unisex bathroom solution (male)

```
turnstile.wait()
    maleSwitch.lock(empty)
turnstile.signal()

    maleMultiplex.wait()
        # bathroom code here
    maleMultiplex.signal()

maleSwitch.unlock (empty)
```

As long as there are men in the room, new arrivals will pass through the turnstile and enter. If there are women in the room when a male arrives, the male will block inside the turnstile, which will bar all later arrivals (male and female) from entering until the current occupants leave. At that point the male in the turnstile enters, possibly allowing additional males to enter.

The female code is similar, so if there are men in the room an arriving female will get stuck in the turnstile, barring additional men.

This solution may not be efficient. If the system is busy, then there will often be several threads, male and female, queued on the turnstile. Each time `empty` is signaled, one thread will leave the turnstile and another will enter. If the new thread is the opposite gender, it will promptly block, barring additional threads. Thus, there will usually be only 1-2 threads in the bathroom at a time, and the system will not take full advantage of the available concurrency.

6.4 Baboon crossing problem

This problem is adapted from Tanenbaum's *Operating Systems: Design and Implementation* [12]. There is a deep canyon somewhere in Kruger National Park, South Africa, and a single rope that spans the canyon. Baboons can cross the canyon by swinging hand-over-hand on the rope, but if two baboons going in opposite directions meet in the middle, they will fight and drop to their deaths. Furthermore, the rope is only strong enough to hold 5 baboons. If there are more baboons on the rope at the same time, it will break.

Assuming that we can teach the baboons to use semaphores, we would like to design a synchronization scheme with the following properties:

- Once a baboon has begun to cross, it is guaranteed to get to the other side without running into a baboon going the other way.
- There are never more than 5 baboons on the rope.

- A continuing stream of baboons crossing in one direction should not bar baboons going the other way indefinitely (no starvation).

I will not include a solution to this problem for reasons that should be clear.

6.5 The Modus Hall Problem

This problem was written by Nathan Karst, one of the Olin students living in Modus Hall[2] during the winter of 2005.

> After a particularly heavy snowfall this winter, the denizens of Modus Hall created a trench-like path between their cardboard shantytown and the rest of campus. Every day some of the residents walk to and from class, food and civilization via the path; we will ignore the indolent students who chose daily to drive to Tier 3. We will also ignore the direction in which pedestrians are traveling. For some unknown reason, students living in West Hall would occasionally find it necessary to venture to the Mods.
>
> Unfortunately, the path is not wide enough to allow two people to walk side-by-side. If two Mods persons meet at some point on the path, one will gladly step aside into the neck high drift to accommodate the other. A similar situation will occur if two ResHall inhabitants cross paths. If a Mods heathen and a ResHall prude meet, however, a violent skirmish will ensue with the victors determined solely by strength of numbers; that is, the faction with the larger population will force the other to wait.

This is similar to the Baboon Crossing problem (in more ways than one), with the added twist that control of the critical section is determined by majority rule. This has the potential to be an efficient and starvation-free solution to the categorical exclusion problem.

Starvation is avoided because while one faction controls the critical section, members of the other faction accumulate in queue until they achieve a majority. Then they can bar new opponents from entering while they wait for the critical section to clear. I expect this solution to be efficient because it will tend to move threads through in batches, allowing maximum concurrency in the critical section.

Puzzle: write code that implements categorical exclusion with majority rule.

[2]Modus Hall is one of several nicknames for the modular buildings, aka Mods, that some students lived in while the second residence hall was being built.

6.5.1 Modus Hall problem hint

Here are the variables I used in my solution.

Modus problem hint

```
heathens = 0
prudes = 0
status = 'neutral'
mutex = Semaphore(1)
heathenTurn = Semaphore(1)
prudeTurn = Semaphore(1)
heathenQueue = Semaphore(0)
prudeQueue = Semaphore(0)
```

`heathens` and `prudes` are counters, and `status` records the status of the field, which can be 'neutral', 'heathens rule', 'prudes rule', 'transition to heathens' or 'transition to prudes'. All three are protected by `mutex` in the usual scoreboard pattern.

`heathenTurn` and `prudeTurn` control access to the field so that we can bar one side or the other during a transition.

`heathenQueue` and `prudeQueue` are where threads wait after checking in and before taking the field.

6.5.2 Modus Hall problem solution

Here is the code for heathens:

Modus problem solution

```
heathenTurn.wait()
heathenTurn.signal()

mutex.wait()
heathens++

if status == 'neutral':
    status = 'heathens rule'
    mutex.signal()
elif status == 'prudes rule':
    if heathens > prudes:
        status = 'transition to heathens'
        prudeTurn.wait()
    mutex.signal()
    heathenQueue.wait()
elif status == 'transition to heathens':
    mutex.signal()
    heathenQueue.wait()
else
    mutex.signal()

# cross the field

mutex.wait()
heathens--

if heathens == 0:
    if status == 'transition to prudes':
        prudeTurn.signal()
    if prudes:
        prudeQueue.signal(prudes)
        status = 'prudes rule'
    else:
        status = 'neutral'

if status == 'heathens rule':
    if prudes > heathens:
        status = 'transition to prudes'
        heathenTurn.wait()

mutex.signal()
```

As each student checks in, he has to consider the following cases:

- If the field is empty, the student lays claim for the heathens.
- If the heathens currently in charge, but the new arrival has tipped the balance, he locks the prude turnstile and the system switches to transition mode.
- If the prudes in charge, but the new arrival doesn't tip the balance, he joins the queue.
- If the system is transitioning to heathen control, the new arrival joins the queue.
- Otherwise we conclude that either the heathens are in charge, or the system is transitioning to prude control. In either case, this thread can proceed.

Similarly, as each student checks out, she has to consider several cases.

- If she is the last heathen to check out, she has to consider the following:
  - If the system is in transition, that means that the prude turnstile is locked, so she has to open it.
  - If there are prudes waiting, she signals them and updates `status` so the prudes are in charge. If not, the new status is 'neutral'.
- If she is not the last heathen to check out, she still has to check the possibility that her departure will tip the balance. In that case, she closes the heathen turnstile and starts the transition.

One potential difficulty of this solution is that any number of threads could be interrupted at Line 3, where they would have passed the turnstile but not yet checked in. Until they check in, they are not counted, so the balance of power may not reflect the number of threads that have passed the turnstile. Also, a transition ends when all the threads that have checked in have also checked out. At that point, there may be threads (of both types) that have passed the turnstile.

These behaviors may affect efficiency—this solution does not guarantee maximum concurrency—but they don't affect correctness, if you accept that "majority rule" only applies to threads that have registered to vote.

Chapter 7

Not remotely classical problems

7.1 The sushi bar problem

This problem was inspired by a problem proposed by Kenneth Reek [9]. Imagine a sushi bar with 5 seats. If you arrive while there is an empty seat, you can take a seat immediately. But if you arrive when all 5 seats are full, that means that all of them are dining together, and you will have to wait for the entire party to leave before you sit down.

Puzzle: write code for customers entering and leaving the sushi bar that enforces these requirements.

7.1.1 Sushi bar hint

Here are the variables I used:

Sushi bar hint

```
eating = waiting = 0
mutex = Semaphore(1)
block = Semaphore(0)
must_wait = False
```

`eating` and `waiting` keep track of the number of threads sitting at the bar and waiting. `mutex` protects both counters. `must_wait` indicates that the bar is (or has been) full, so incoming customers have to block on `block`.

7.1.2 Sushi bar non-solution

Here is an incorrect solution Reek uses to illustrate one of the difficulties of this problem.

Sushi bar non-solution

```
mutex.wait()
if must_wait:
    waiting += 1
    mutex.signal()
    block.wait()

    mutex.wait()      # reacquire mutex
    waiting -= 1

eating += 1
must_wait = (eating == 5)
mutex.signal()

# eat sushi

mutex.wait()
eating -= 1
if eating == 0:
    n = min(5, waiting)
    block.signal(n)
    must_wait = False
mutex.signal()
```

Puzzle: what's wrong with this solution?